FATEC OSASCO PREFEITO HIRANT SANAZAR

Desenvolvimento de Software Multiplataforma

TROCAÍ

Aplicativo de Troca de Habilidades entre Moradores de Condomínios

Entrega 4 — Implementação do núcleo do aplicativo

Objetivo específico 4 · prazo 07/11/2026

Jefferson de Padua Coelho Ribeiro — RA: 2161392413009

Karine Fernandes e Silva — RA: 2161392413030

Sarah Quinteiro Limeira da Silva — RA: 2161392413027

Orientação: Professor Elias Carneiro de Oliveira

Osasco · 2026 / 6° semestre

1. Resumo

Condomínios residenciais reúnem, em poucos metros quadrados, uma diversidade enorme de competências que permanece invisível: moradores contratam serviços externos e pagam por aulas particulares sem saber que o vizinho do andar de cima domina exatamente aquilo. O Trocaí é um aplicativo mobile que torna esse conhecimento visível e trocável. O morador cadastra o que sabe fazer e o que precisa, encontra vizinhos compatíveis dentro do próprio condomínio e combina o encontro nas áreas comuns por um chat interno. As trocas são registradas por créditos virtuais, chamados Elos, sem qualquer envolvimento de dinheiro real: quem ensina acumula, quem aprende gasta. Um painel web complementar entrega ao síndico o ranking de moradores ativos, a gestão de avisos e bloqueios e a Árvore de Talentos, visualização que revela quais habilidades estão em falta na comunidade. O levantamento de requisitos foi conduzido pela metodologia Lean Inception, percorrendo visão de produto, personas, jornada do usuário, brainstorming de funcionalidades, revisão técnica e sequenciamento em ondas. O desenvolvimento prossegue em entregas incrementais, das quais esta documenta a de número 4.

Palavras-chave: Aplicativo Móvel. Mobile Development. Flutter. Economia Colaborativa. Lean Inception. Solução Tecnológica.

2. Introdução

2.1 Tema e justificativa

Viver em condomínio vertical no Brasil significa, hoje, dividir parede com dezenas de pessoas e não conhecer nenhuma delas. Segundo o Instituto Nacional de Condomínios, existem mais de 520 mil condomínios no país, e em cada um deles convivem professores, desenvolvedores, cozinheiros, costureiras, músicos, estudantes e aposentados com décadas de ofício. Esse conjunto de competências é um recurso ocioso: ninguém sabe que ele existe e, por isso, ninguém o utiliza.

O resultado prático é duplo. De um lado, o morador recorre ao mercado externo e paga por um serviço que o vizinho faria de graça; de outro, quem tem tempo e conhecimento disponíveis, como aposentados e estudantes, não encontra canal para oferecê-los nem reconhecimento por fazê-lo. O grupo de WhatsApp, única ferramenta de comunicação da maioria dos prédios, não resolve: pedidos se perdem no volume de mensagens, não há histórico, não há moderação e não existe mecanismo de reciprocidade que garanta que quem ajuda também será ajudado.

A solução precisa ser móvel por três razões. A primeira é de contexto de uso: o pedido nasce no momento em que o problema aparece, longe do computador, e o encontro acontece em uma área comum do próprio prédio. A segunda é de alcance: o smartphone é o único dispositivo que praticamente todos os moradores possuem, o que permite incluir perfis de baixa afinidade digital. A terceira é de notificação: a economia de trocas só funciona se a resposta a um pedido chegar rápido, e apenas o aplicativo entrega isso.

A relevância social está em transformar vizinhança em comunidade sem exigir dinheiro de ninguém, aproveitando um recurso que já existe e hoje se perde. A relevância técnica está no desafio de construir uma economia interna confiável, em que o saldo de créditos nunca divirja, e uma camada de visualização capaz de revelar lacunas de competência a quem administra o condomínio.

2.2 Problema de pesquisa

Como viabilizar, por meio de um aplicativo móvel, a troca de habilidades entre moradores de um mesmo condomínio, de forma confiável e recíproca, sem envolver dinheiro real e sem depender da coincidência entre quem oferece e quem procura?

2.2.1 O produto é, não é, faz, não faz

O produto É	O produto NÃO É
Um ecossistema digital: aplicativo mobile e painel web	Uma rede social aberta ao público
Uma rede fechada e verificada por condomínio	Um marketplace de serviços pagos
Um sistema de economia colaborativa	Uma plataforma de contratação de freelancers
Uma moeda de reciprocidade em créditos virtuais	Uma criptomoeda ou meio de pagamento
Um catálogo vivo dos talentos do prédio	Um sistema de gestão condominial de boletos e atas
Comunitário, local, gratuito e voluntário	Um aplicativo de relacionamento
Simples e acessível, inclusive para idosos	Complexo ou voltado a público técnico
Moderado e transparente para a administração	Anônimo ou obrigatório

O produto FAZ	O produto NÃO FAZ
Cadastra e valida moradores por unidade	Não processa pagamentos nem converte créditos em dinheiro
Divulga as habilidades que cada morador oferece	Não emite nota fiscal nem contrato de prestação de serviço
Registra as necessidades que cada morador procura	Não permite trocas entre condomínios diferentes
Conecta oferta e demanda dentro do mesmo condomínio	Não verifica antecedentes nem certifica profissionalmente
Credita e debita Elos ao concluir a troca	Não garante a qualidade técnica do serviço prestado
Retém os créditos em custódia até a dupla confirmação	Não permite transferir créditos fora de uma troca registrada
Aproxima vizinhos pelo chat interno	Não expõe telefone, e-mail ou número do apartamento
Agenda encontros nas áreas comuns	Não reserva nem controla a ocupação dos espaços
Avalia reputação de forma mútua	Não julga nem arbitra conflitos automaticamente
Publica os avisos do síndico	Não substitui o aplicativo oficial da administradora
Bloqueia trocas em períodos definidos pela administração	Não cancela trocas já aceitas durante o bloqueio
Revela as habilidades em falta pela Árvore de Talentos	Não vende dados dos moradores nem exibe publicidade

2.2.2 Personas

Foram construídas três personas, escolhidas para cobrir os dois lados da economia de trocas e também o público do painel web. O produto só funciona se os três perfis forem atendidos ao mesmo tempo: sem quem oferta não há o que trocar, sem quem solicita os créditos não circulam, e sem o síndico o condomínio não adota a ferramenta.

Camila Duarte, 29 anos, analista de marketing — a Nova no Prédio

"Moro aqui há dois anos e não sei o nome de ninguém do meu andar."

Dimensão	Descrição
Perfil	Mora sozinha no apartamento 74B, trabalha em home office quatro dias por semana. Chegou ao condomínio na pandemia e nunca criou vínculo com os vizinhos. Renda estável, mas orçamento

Dimensão	Descrição
	apertado depois do aumento do aluguel.
Comportamentos	Usa o celular o dia inteiro e resolve quase tudo por aplicativo. Está no grupo de WhatsApp do prédio, mas deixou silenciado. Contrata serviços pelo preço, sem indicação. Evita ligar para desconhecidos. Revisa currículos de amigos de graça, por hábito.
Necessidades	Consertar o notebook sem pagar visita técnica cara. Aprender violão. Sentir-se parte do lugar onde mora. Retribuir o que recebe, para não ficar devendo favor.
Dores	Pediu indicação no grupo e recebeu quinze respostas desencontradas. Receia deixar estranho entrar no apartamento. Acha que não tem nada a oferecer. Desconfia de aplicativo que pede telefone e endereço no cadastro.
Uso do produto	Aplicativo. É o perfil que mais solicita no começo e passa a ofertar quando percebe que a habilidade dela tem demanda.

Antônio Ribeiro, 63 anos, aposentado — o Vizinho Prestativo

"Eu conserto, só não sei mexer nesses aplicativos de hoje."

Dimensão	Descrição
Perfil	Ex-técnico em eletrônica por 34 anos, casado, mora no apartamento 22A há quinze anos. Conhece metade do prédio pelo nome. Aposentadoria apertada.
Comportamentos	Usa o celular para WhatsApp e chamadas com os netos. Aumenta a fonte do sistema ao máximo e ainda tem dificuldade com botões pequenos. Abandona cadastros com mais de três ou quatro campos. Passa a manhã na área comum. Já consertou ventilador, chuveiro e roteador de vizinhos sem cobrar.
Necessidades	Continuar sendo útil e reconhecido. Conseguir ajuda com burocracia digital. Ocupar o tempo livre com propósito. Não depender de favor.
Dores	Sente que ficou invisível depois de se aposentar. Já foi vítima de golpe por link e desconfia de tudo. Cansou de ajudar sempre e nunca receber. Desiste na primeira tela quando a interface tem ícones pequenos.
Uso do produto	Aplicativo. É o ofertante que sustenta a economia de créditos no início; se ele não entra, o sistema não sai do lugar.

Rita Menezes, 45 anos, síndica — a Gestora Sobrecarregada

"Eu decido no achismo porque não tenho dado nenhum sobre o que acontece aqui."

Dimensão	Descrição
Perfil	Advogada, síndica há dois anos, concilia o mandato com trabalho em tempo integral. Administra 96 unidades com salão de festas e espaço gourmet subutilizados.
Comportamentos	Prefere resolver as coisas do condomínio pelo navegador. Apaga incêndio no grupo várias vezes por semana. Prepara a prestação de contas na véspera da assembleia. Publica avisos impressos no elevador.
Necessidades	Reduzir atrito entre moradores. Aumentar o uso das áreas comuns. Ter números concretos para a assembleia. Suspender atividades em períodos críticos. Descobrir o que a comunidade precisa.
Dores	Não sabe quais habilidades existem no prédio. Teme ser responsabilizada se uma troca der errado. Já tentou organizar mutirões e ninguém apareceu. Não tem tempo para moderar mais um canal.
Uso do produto	Painel web. É quem aprova cadastros, publica avisos, bloqueia períodos e consulta a Árvore de Talentos.

2.2.3 Jornada do usuário

As jornadas descrevem o percurso de cada persona em etapas cronológicas, com a emoção de cada momento, as dores críticas e o ponto exato em que o produto entra em cena. A coluna de marcação identifica onde está a dor e onde o valor se realiza.

Jornada 1 — Camila: conserta o notebook

#	Etapa	Marcação
1	acorda e começa a trabalhar em casa	neutra
2	o notebook trava no meio da manhã	neutra
3	perde a manhã tentando resolver sozinha	DOR CRÍTICA
4	pede indicação no grupo do prédio	neutra
5	recebe quinze respostas desencontradas	DOR CRÍTICA
6	abre o Trocaí	entrada do produto
7	busca conserto de computador no prédio	neutra
8	encontra o Seu Antônio, do 22A	alívio
9	combina o encontro pelo chat	alívio
10	paga em créditos e ganha um vizinho	MOMENTO DE VALOR

Contexto de uso: O aplicativo é acionado em casa, no meio do expediente, logo após a tentativa frustrada pelo canal informal do condomínio.

Jornada 2 — Seu Antônio: ser encontrado pelos vizinhos

#	Etapa	Marcação
1	acorda cedo e não tem compromisso	neutra
2	passa a manhã na área comum	neutra
3	um vizinho pede ajuda no corredor	neutra
4	conserta o ventilador e não cobra nada	alívio
5	ninguém registra o que ele fez	DOR CRÍTICA
6	precisa de ajuda e não sabe a quem pedir	DOR CRÍTICA
7	o filho instala o Trocaí para ele	entrada do produto
8	cadastra conserto de eletrônicos	neutra
9	recebe pedidos de vizinhos que não conhecia	alívio
10	usa os créditos para pedir ajuda com o celular	MOMENTO DE VALOR

Contexto de uso: O aplicativo entra como registro do que antes era favor informal, convertendo reconhecimento em saldo utilizável.

Jornada 3 — Rita: saber o que falta no condomínio

#	Etapa	Marcação
1	chega em casa depois do trabalho	neutra
2	abre o grupo do prédio com 200 mensagens	DOR CRÍTICA
3	apaga mais um incêndio entre vizinhos	DOR CRÍTICA
4	passa pelo salão de festas vazio	neutra
5	monta a prestação de contas sem dado nenhum	DOR CRÍTICA
6	acessa o painel do Trocaí	entrada do produto
7	confere o ranking de moradores ativos	neutra
8	a Árvore de Talentos mostra o que falta	alívio
9	publica um aviso e bloqueia a semana da reforma	alívio
10	leva números concretos para a assembleia	MOMENTO DE VALOR

Contexto de uso: Diferentemente das anteriores, esta jornada percorre o painel web e não gera troca alguma; ela alimenta decisão administrativa.

Observação transversal: nas três jornadas as dores críticas se concentram antes da entrada do produto e as emoções positivas só aparecem depois dela. Uma segunda observação, menos confortável, é que nenhuma das jornadas acontece com frequência alta, o que levou à inclusão de oficinas abertas no escopo das ondas seguintes.

2.3 Objetivos

2.3.1 Objetivo geral

Desenvolver um aplicativo mobile, acompanhado de um painel web administrativo, que viabilize a troca de habilidades entre moradores de um mesmo condomínio por meio de um sistema de créditos virtuais sem valor monetário, promovendo economia colaborativa, aproveitamento de competências ociosas e fortalecimento dos laços comunitários.

2.3.2 Objetivos de Desenvolvimento Sustentável e benefícios à sociedade

ODS	Alinhamento
ODS 11 — Cidades e Comunidades Sustentáveis	Alinhamento principal. Fortalece laços de vizinhança, aumenta o uso de espaços comuns ociosos e estimula a resolução local de necessidades.
ODS 12 — Consumo e Produção Responsáveis	Substitui a contratação externa pelo aproveitamento de competências já existentes no prédio, reduzindo consumo desnecessário.
ODS 4 — Educação de Qualidade	Boa parte das trocas é de ensino entre pares, ampliando o acesso a conhecimento sem barreira financeira.
ODS 10 — Redução das Desigualdades	A moeda do sistema é o tempo, não o dinheiro, o que dá poder de troca igual a quem tem renda alta e a quem tem renda baixa.

Benefícios à sociedade:

- Para o morador: economia direta em serviços cotidianos, acesso a aprendizado gratuito e redução do isolamento social

- Para o condomínio: ocupação das áreas comuns, redução de conflitos e informação concreta para a assembleia
- Para a comunidade ampliada: valorização de saberes não formais e um modelo replicável de economia solidária

2.3.3 Objetivos específicos — cronograma

As seis entregas abaixo organizam o desenvolvimento até o fim do semestre. A coluna de situação indica o andamento no momento desta entrega.

Data	Objetivo específico	Situação
17/10/2026	Formalizar os requisitos funcionais e não funcionais em histórias de usuário com critérios de aceitação e elaborar o diagrama de casos de uso dos dois perfis	Entregue
24/10/2026	Desenvolver a interface no Figma, com protótipo navegável das telas principais, atendendo aos requisitos de acessibilidade levantados na persona de baixa afinidade digital	Entregue
31/10/2026	Definir a base de dados, entregando o modelo entidade-relacionamento, o dicionário de dados e as regras de integridade da carteira de créditos	Entregue
07/11/2026	Implementar o núcleo do aplicativo em Flutter, integrado a uma API em Node.js com PostgreSQL, contemplando as funcionalidades da Onda 1	Esta entrega
14/11/2026	Realizar testes funcionais documentados e validação com usuários em um condomínio piloto, registrando defeitos, feedback e taxa de conclusão das trocas	Previsto
28/11/2026	Criar o pitch de apresentação e concluir a documentação do projeto, consolidando resultados, limitações e propostas de evolução	Previsto

3. Fundamentação Teórica

3.1 Desenvolvimento nativo, híbrido e multiplataforma

O desenvolvimento nativo consiste em construir o aplicativo na linguagem e no SDK próprios de cada plataforma, Kotlin e Jetpack Compose no Android e Swift e SwiftUI no iOS. Entrega o melhor desempenho e acesso imediato a recursos novos do sistema, mas exige duas bases de código distintas e, consequentemente, duas equipes ou o dobro do tempo. O desenvolvimento híbrido tradicional, baseado em WebView, resolve o custo mas penaliza a fluidez da interface. A abordagem multiplataforma compilada, adotada por Flutter e React Native, ocupa o meio-termo: uma única base de código gera artefatos para Android e iOS com desempenho próximo ao nativo. Para um projeto acadêmico com equipe reduzida e prazo de um semestre, essa é a escolha tecnicamente justificada.

3.2 Flutter e Dart

O Flutter é um framework de código aberto mantido pelo Google que compila Dart para código nativo ARM e desenha a própria interface por meio do motor gráfico Skia, em vez de delegar aos componentes do sistema. Essa decisão arquitetural garante que a aparência seja idêntica nas duas plataformas e dá controle fino sobre acessibilidade, ponto relevante para este projeto por causa da persona de baixa afinidade digital. O modelo de widgets declarativo e o recurso de recarga a quente reduzem o ciclo de iteração durante o desenvolvimento.

3.3 Interface e experiência do usuário em aplicações móveis

A literatura de usabilidade móvel converge em alguns princípios que orientam este projeto. As diretrizes do Material Design recomendam área mínima de toque de 48 por 48 pixels independentes de densidade, o que afeta diretamente a usabilidade para usuários idosos. As WCAG 2.1, no nível AA, estabelecem razão de contraste mínima de 4,5 para 1 em texto comum e exigem que o conteúdo permaneça legível com a fonte ampliada em até 200 por cento. A heurística de Nielsen sobre visibilidade do estado do sistema justifica a exibição permanente do saldo de créditos, e a de reconhecimento em vez de memorização sustenta a navegação por categorias além da busca textual pura.

3.4 Persistência de dados em aplicações móveis

Aplicações móveis costumam combinar armazenamento local e remoto. O SQLite, embarcado no dispositivo, atende cache e funcionamento offline. No lado remoto há duas famílias. Soluções NoSQL gerenciadas, como o Firebase Firestore, oferecem sincronização em tempo real e baixo custo inicial de configuração, mas não garantem transações multi-documento com a mesma robustez de um banco relacional. Bancos relacionais, como o PostgreSQL, oferecem transações ACID, restrições de integridade e consistência forte. Como a carteira de créditos do Trocaí não pode divergir em nenhuma hipótese, uma transferência de Elos precisa ser atômica: ou debita e credita, ou não faz nada. Esse requisito pesa a favor do modelo relacional para o núcleo da aplicação.

3.5 Economia colaborativa e bancos de tempo

O conceito de banco de tempo, formalizado por Edgar Cahn na década de 1980, propõe uma moeda complementar em que a unidade de valor é a hora de serviço prestado, igual para qualquer tipo de trabalho. A premissa é que a reciprocidade registrada reduz o constrangimento de pedir ajuda e torna sustentável o que, informalmente, se esgota. A literatura sobre essas iniciativas também registra o principal fator de fracasso: a falta de liquidez. Em comunidades pequenas, oferta e demanda raramente coincidem no tempo, e participantes acumulam créditos sem conseguir gastá-los. Esse achado foi incorporado ao projeto e motivou a proposta de oficinas abertas, em que uma única oferta atende vários moradores simultaneamente.

3.6 Lean Inception

A Lean Inception, sistematizada por Paulo Caroli, é um workshop colaborativo que combina Design Thinking e Lean Startup para definir o produto mínimo viável. Estrutura-se em atividades sequenciais que partem da visão de produto e avançam por É-Não É-Faz-Não Faz, objetivos, personas, jornadas, brainstorming de funcionalidades, revisão técnica e de negócio, sequenciador e canvas do MVP. A metodologia foi adotada como instrumento de levantamento de requisitos porque garante rastreabilidade: cada funcionalidade proposta tem origem em uma oportunidade identificada em uma jornada, que por sua vez pertence a uma persona definida.

4. Metodologia

4.1 Tipo de pesquisa

Trata-se de pesquisa aplicada, de natureza exploratória e abordagem qualitativa, materializada no desenvolvimento de um aplicativo mobile como estudo de caso.

A fase de levantamento de requisitos foi conduzida pela metodologia Lean Inception, em sessões remotas facilitadas com uso de quadro digital colaborativo. Cada atividade seguiu o mesmo protocolo:

contextualização pelo facilitador, geração silenciosa de ideias com tempo cronometrado para evitar que apenas os participantes mais comunicativos definissem o resultado, agrupamento por afinidade e consolidação por consenso. Nas decisões de divergência, adotou-se o critério de que questões técnicas são decididas pelos desenvolvedores e questões de valor pelo responsável pelo produto.

A fase de construção adota desenvolvimento incremental organizado em seis entregas quinzenais, conforme o cronograma da seção 2.3.3, com validação ao final de cada uma.

4.2 Ferramentas e tecnologias

Camada	Tecnologia	Justificativa
Aplicativo mobile	Flutter com Dart	Base de código única para Android e iOS, com controle fino sobre acessibilidade
Painel web	React com TypeScript	Ecossistema maduro e bibliotecas de visualização de dados
Backend	Node.js com Express, API REST	Produtividade e mesma linguagem do front web
Banco de dados	PostgreSQL	Transações ACID, indispensáveis para a carteira de créditos
Testes	Jest	Execução rápida e relatório de cobertura integrado
Notificações	Firebase Cloud Messaging	Push gratuito
Design	Figma e FigJam	Protótipos, design system e quadro colaborativo
Versionamento	Git e GitHub Actions	Fluxo de revisão por pares e integração contínua

4.3 Levantamento de requisitos

4.3.1 Requisitos funcionais

Código	Descrição
RF01	Cadastrar conta informando nome, unidade e código do condomínio
RF02	Validar o morador por aprovação do síndico antes de liberar o acesso
RF03	Cadastrar habilidades oferecidas, com categoria, descrição e custo em créditos
RF04	Cadastrar necessidades procuradas
RF05	Buscar habilidades por palavra-chave e filtrar por categoria, bloco e disponibilidade
RF06	Enviar, aceitar, recusar e cancelar solicitações de troca
RF07	Reter os créditos em custódia entre o aceite e a conclusão da troca
RF08	Exigir confirmação das duas partes para dar a troca por concluída
RF09	Transferir os créditos automaticamente após a dupla confirmação
RF10	Exibir o saldo na tela inicial e o extrato de entradas e saídas
RF11	Bloquear novas solicitações quando o saldo estiver abaixo do limite de dívida
RF12	Trocar mensagens por chat interno vinculado a cada troca
RF13	Agendar data, horário e área comum do encontro
RF14	Avaliar o vizinho com nota e comentário após a conclusão
RF15	Denunciar conduta inadequada e permitir suspensão do usuário
RF16	Notificar novas solicitações, mensagens e avisos

Código	Descrição
RF17	Aprovar ou recusar cadastros de moradores pelo painel web
RF18	Publicar e agendar avisos para todo o condomínio
RF19	Bloquear novas trocas em um intervalo de datas, preservando as já aceitas
RF20	Exibir o ranking de moradores mais ativos e a Árvore de Talentos com as habilidades em falta
RF21	Exibir o perfil público do vizinho, com habilidades, avaliação média e trocas concluídas
RF22	Listar as solicitações recebidas em ordem de chegada
RF23	Marcar períodos de indisponibilidade no próprio perfil
RF24	Ordenar os resultados da busca por avaliação e destacar novos ofertantes
RF25	Informar a duração estimada da troca no momento da solicitação
RF26	Conceder selos por marcos de participação

4.3.2 Requisitos não funcionais

Código	Descrição
RNF01	Compatibilidade: Android 8.0 ou superior e iOS 13 ou superior; painel web responsivo
RNF02	Desempenho: resposta da API abaixo de 500 ms em 95% das requisições; mensagem no chat abaixo de 1 segundo
RNF03	Confiabilidade: movimentação de créditos em transação atômica, com registro imutável, sem divergência de saldo
RNF04	Segurança: senhas com hash, autenticação por token com expiração e acesso restrito a moradores validados
RNF05	Privacidade e LGPD: consentimento explícito, exibição pública limitada a nome e bloco, contato só pelo chat interno, direito à exclusão com anonimização
RNF06	Acessibilidade: WCAG 2.1 nível AA, leitor de tela, fonte ampliada e área de toque mínima de 48 por 48 dp
RNF07	Usabilidade: cadastro completo em até três minutos; fluxo principal em no máximo quatro toques
RNF08	Disponibilidade: 99% no intervalo das 6h às 23h
RNF09	Restrição de domínio: nenhuma transação financeira real; créditos não podem ser comprados, vendidos ou convertidos
RNF10	Escalabilidade: modelo de dados multi-tenant, preparado para vários condomínios de forma isolada
RNF11	Manutenibilidade: cobertura mínima de 60% de testes automatizados no backend e padronização por linter

4.3.3 Regras de negócio da economia de Elos

Código	Descrição
RN01	1 Elo equivale a 30 minutos de dedicação. É referência sugerida, negociável entre as partes
RN02	Todo novo morador aprovado recebe 3 Elos de boas-vindas
RN03	Elos não têm valor monetário e não podem ser comprados, vendidos nem convertidos em dinheiro
RN04	O débito e o crédito só ocorrem após a confirmação de ambas as partes
RN05	O saldo pode ficar negativo até o limite de -3 Elos; abaixo disso o morador só pode ofertar
RN06	Elos ficam em custódia desde o aceite da troca até a conclusão ou o cancelamento
RN07	Cancelamento antes do encontro devolve integralmente os Elos retidos
RN08	Divergência na confirmação abre um chamado para mediação do síndico

Código	Descrição
RN09	Elos são pessoais e intransferíveis fora de uma troca registrada
RN10	Durante um período bloqueado, novas solicitações ficam indisponíveis; trocas aceitas permanecem válidas
RN11	Teto de 10 trocas concluídas por morador por mês, para evitar profissionalização disfarçada

5. Desenvolvimento (resultados e discussões)

Esta seção apresenta o resultado do objetivo específico 4, previsto para 07/11/2026: implementação do núcleo do aplicativo. A consolidação de todas as entregas parciais em um documento único está prevista para a entrega de 28/11/2026.

5.1 Implementação

A implementação cobre as funcionalidades da Onda 1 definida no sequenciador, contemplando as cinco áreas previstas para esta entrega: cadastro de morador com validação pelo síndico, publicação de habilidades, busca com filtros, solicitação de troca e movimentação de créditos. Acompanham o núcleo o perfil público, o registro de necessidades, o aceite, a conclusão com dupla confirmação, a avaliação e o chat simples.

5.1.1 Arquitetura da solução

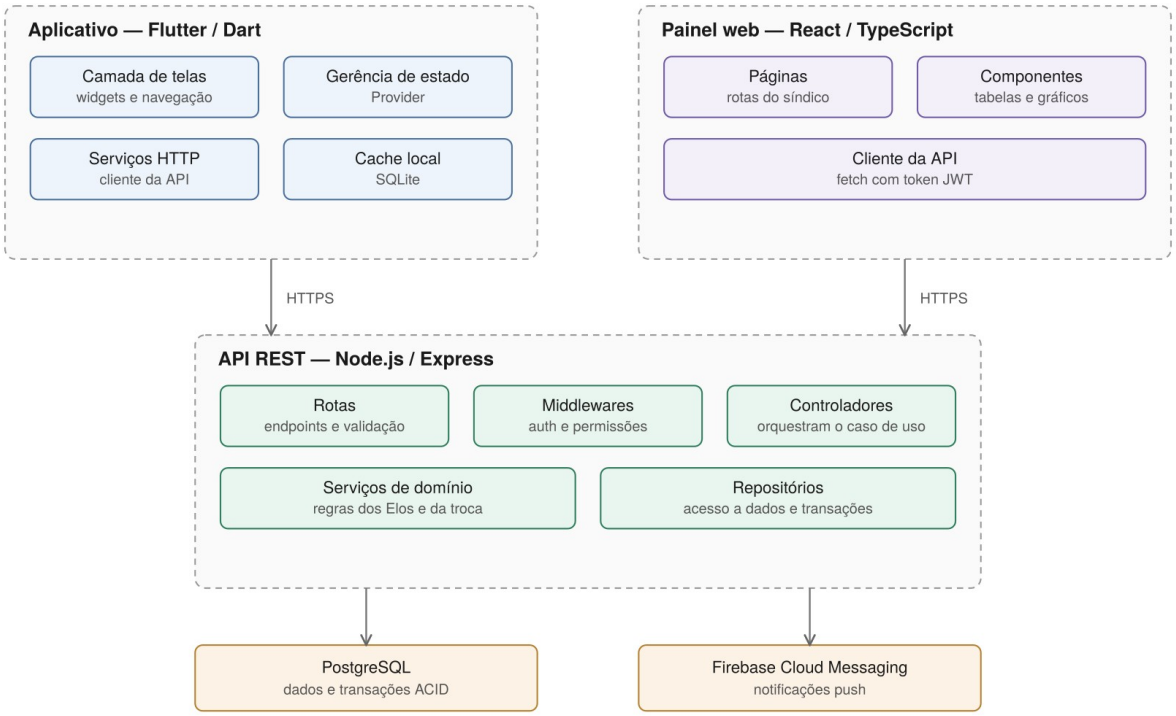


Figura 1 — Arquitetura em camadas do Trocaí

A API concentra toda a regra de negócio. O aplicativo e o painel web são clientes sem lógica de domínio, o que garante que as regras dos Elos sejam idênticas nas duas interfaces e que uma eventual terceira interface não precise reimplementá-las. A separação entre serviços de domínio e repositórios mantém as regras testáveis sem banco de dados.

5.1.2 Organização do código

```
trocai-api/
  src/
    rotas/          definição dos endpoints e validação de entrada
    middlewares/    autenticação JWT, permissões, tratamento de erro
    controladores/  orquestram um caso de uso por método
    servicos/       regras de negócio (TrocaService, CarteiraService)
```

```
    repositorios/    acesso ao banco e controle de transação
    db/              pool de conexões e migrações
  testes/
    unitarios/       serviços de domínio, sem banco
    integracao/      endpoints contra banco de teste

trocai-app/
  lib/
    telas/           uma pasta por tela do inventário de telas
    componentes/     widgets reutilizáveis do design system
    estado/          providers
    servicos/        cliente HTTP e armazenamento local
    modelos/         classes de domínio
```

5.1.3 Endpoints da API

São 32 endpoints sob o prefixo /api. Fora /saude, /auth/cadastro e /auth/login, toda rota exige o cabeçalho Authorization com o token da sessão. As rotas sob /admin exigem papel de síndico, verificado no próprio token.

Método e rota	Função	Caso de uso
GET /saude	Verificação de disponibilidade	—
POST /auth/cadastro	Cria conta pendente de aprovação	UC01
POST /auth/login	Autentica e devolve token JWT	—
GET /auth/eu	Dados do morador autenticado	—
GET /categorias	Categorias para o filtro da busca	UC04
GET /moradores/:id	Perfil público do vizinho	UC04
GET /habilidades?q=&categoria=	Busca com filtros	UC04
GET /habilidades/minhas	Habilidades que eu publiquei	UC02
POST /habilidades	Publica habilidade oferecida	UC02
DELETE /habilidades/:id	Despublica uma habilidade	UC02
GET /necessidades	O que o condomínio procura	UC03
POST /necessidades	Registra necessidade procurada	UC03
GET /trocas?papel=&status=	Lista trocas do morador autenticado	UC06
POST /trocas	Solicita troca a um vizinho	UC05
PATCH /trocas/:id/aceitar	Aceita e reserva os créditos	UC06
PATCH /trocas/:id/recusar	Recusa sem movimentar créditos	UC06
PATCH /trocas/:id/confirmar	Registra confirmação de uma das partes	UC07
PATCH /trocas/:id/cancelar	Cancela e devolve o que estava retido	UC06
GET /trocas/:id/mensagens	Lê o chat da troca	UC08
POST /trocas/:id/mensagens	Envia mensagem no chat da troca	UC08
POST /trocas/:id/avaliacao	Avalia a outra parte	UC09
POST /denuncias	Registra denúncia de conduta	UC16
GET /carteira	Saldo, custódia e extrato	UC10
GET /carteira/auditoria	Confere o saldo contra o livro-razão	UC10

Método e rota	Função	Caso de uso
GET /admin/cadastros	Lista cadastros pendentes	UC11
PATCH /admin/cadastros/:id	Aprova ou recusa cadastro	UC11
PATCH /admin/moradores/:id/suspender	Suspende o acesso de um morador	UC12
GET /admin/painel	Indicadores, ranking e Árvore de Talentos	UC14
POST /admin/bloqueios	Suspende as trocas do condomínio	UC13
PATCH /admin/bloqueios/:id/liberar	Encerra a suspensão	UC13
GET /admin/denuncias	Denúncias em aberto	UC16
PATCH /admin/denuncias/:id	Registra a decisão do síndico	UC15, UC16

Os códigos de resposta seguem um padrão único: 401 para token ausente, inválido, cadastro pendente, acesso suspenso, falta de permissão ou operação fora das próprias trocas; 404 para recurso inexistente; 409 para registro duplicado; 422 para regra de negócio recusada; e 500 para falha interna, cujo detalhe fica no log e não na resposta. Dentro de /api, uma rota inexistente responde 401 a quem não apresentou token, antes de dizer se ela existe: responder 404 permitiria mapear a API de fora.

5.1.4 Implementação da regra crítica

O método abaixo concentra o maior risco técnico do sistema. A confirmação só dispara a transferência quando as duas partes já confirmaram, e toda a operação acontece dentro de uma transação.

```
async confirmarConclusao(trocaId, moradorId) {
  return this.db.transacao(async (tx) => {
    const troca = await tx.trocas.buscarParaAtualizar(trocaId);

    if (troca.status !== 'aceita')
      throw new RegraDeNegocio('A troca não está em andamento.');

    const ehSolicitante = troca.solicitante_id === moradorId;
    const ehOfertante   = troca.ofertante_id === moradorId;
    if (!ehSolicitante && !ehOfertante)
      throw new NaoAutorizado('Você não participa desta troca.');

    await tx.trocas.registrarConfirmacao(trocaId, ehSolicitante);

    const atual = await tx.trocas.buscar(trocaId);
    const ambasConfirmaram = atual.conf_solicitante && atual.conf_ofertante;

    if (!ambasConfirmaram) {
      return { status: 'aguardando_outra_parte' };
    }

    // dupla confirmação: transfere os créditos
    await this.carteira.transferir(tx, {
      de: atual.solicitante_id,
      para: atual.ofertante_id,
      valor: atual.custo_elos,
      trocaId
    });

    await tx.trocas.concluir(trocaId);
    return { status: 'concluida' };
  });
}
```

Qualquer exceção lançada dentro do bloco reverte a transação inteira, incluindo a confirmação já registrada. Isso evita o estado inconsistente em que uma confirmação é gravada mas a transferência falha.

5.1.5 Telas da Onda 1

A implementação desta etapa cobre a API, que concentra toda a regra de negócio; o aplicativo em Flutter consome esses endpoints e, neste momento, está na forma do protótipo navegável definido na entrega anterior. As telas abaixo mostram o fluxo completo de uma troca e documentam o contrato entre as duas pontas: cada informação exibida corresponde a um campo devolvido por um dos endpoints desta entrega.

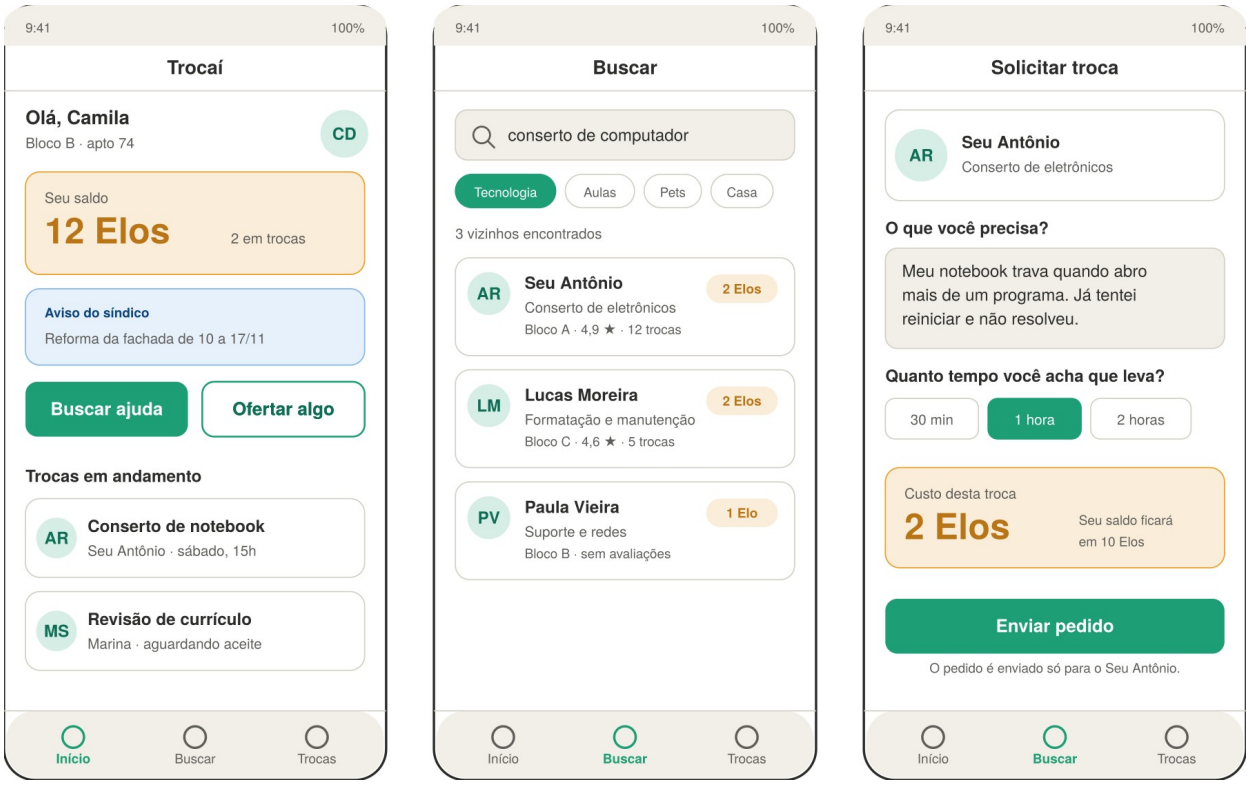


Figura 2 — Tela inicial com o saldo, busca com resultados e pedido de troca

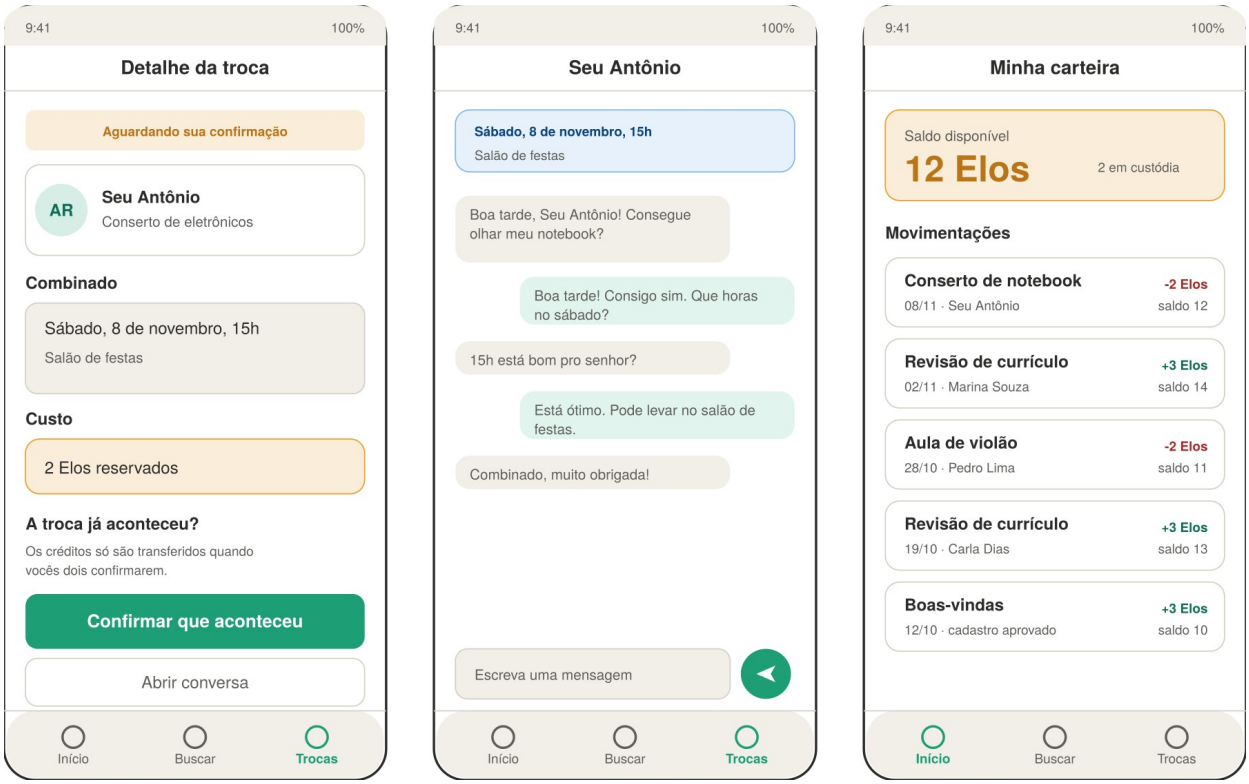


Figura 3 — Troca aguardando confirmação, chat da troca e extrato após a transferência

O extrato da terceira tela da última figura segue o formato devolvido por GET /carteira: tipo do lançamento, valor com sinal e saldo resultante, na ordem em que o livro-razão os grava. Os valores das telas são de um cenário de demonstração; a sequência de débito e crédito que elas representam é a do caso CT03, verificada pela suíte automatizada e reproduzível pelo script de demonstração descrito adiante.

5.1.6 Decisões técnicas registradas

Decisão	Alternativa descartada	Motivo
Regra de negócio apenas na API	Validação duplicada no aplicativo	Garante que app e painel web apliquem a mesma regra; o cliente valida só formato de campo
Chat por requisição periódica na Onda 1	WebSocket desde o início	A versão simples entrega quase todo o valor de experiência com conhecimento técnico verde, conforme a revisão da Atividade 7
Custo em Elos congelado no aceite	Custo lido da habilidade na conclusão	Impede que o ofertante altere o preço depois do combinado
Saldo materializado em coluna	Soma calculada a cada leitura	O saldo é lido em toda abertura do app; a consistência é garantida por transação e auditoria
Busca com índice de texto do PostgreSQL	Serviço de busca dedicado	O volume de um condomínio não justifica infraestrutura adicional

5.1.7 Repositório do código-fonte

O código-fonte do produto está publicado em:

```
https://github.com/FinEnsina/Trocai
```

O repositório reúne, além do código, a documentação de todas as entregas e os artefatos visuais do projeto:

Caminho	Conteúdo
api/src/	Implementação da API: rotas, middlewares, controladores, serviços, repositórios e migrações
api/testes/	Suíte automatizada: testes unitários de domínio e testes de integração dos endpoints
api/scripts/	Script que percorre uma troca inteira pela API, para demonstração
api/README.md	Endpoints, códigos de resposta e decisões técnicas
docs/entregas/	Os documentos das seis entregas em PDF
docs/lean-inception/	Registro das nove atividades do workshop
docs/diagramas/	Casos de uso, navegação, modelo de dados, arquitetura e telas

Para executar:

```
cd api
npm install
cp .env.example .env        # credenciais do PostgreSQL
createdb trocai
npm run migrar              # cria o esquema e os dados de exemplo
npm start                   # http://localhost:3000/api

npm test                    # suíte automatizada completa
npm run demonstracao        # troca inteira, sem precisar de banco
```

O comando npm run demonstracao sobe a API sobre o banco de teste em memória e executa o fluxo completo — cadastro, aprovação pelo síndico, busca, pedido, aceite, conversa, dupla confirmação, extrato, auditoria de integridade e avaliação —, imprimindo o que acontece com os saldos das duas carteiras a cada passo. É a forma de verificar o comportamento do domínio em qualquer máquina, sem instalar PostgreSQL. Os resultados da suíte automatizada estão registrados na seção de testes e validação.

6. Considerações parciais

Esta entrega concluiu o objetivo específico 4, previsto para 07/11/2026: implementação do núcleo do aplicativo. A próxima entrega, prevista para 14/11/2026, tratará de testes e validação.

O documento é cumulativo: cada entrega revisa e consolida as anteriores, de modo que esta versão substitui integralmente as precedentes. A conclusão do trabalho, com avaliação dos objetivos alcançados, limitações e trabalhos futuros, será apresentada na entrega de 28/11/2026.

Etapas restantes:

Prazo	Objetivo
14/11/2026	Testes e validação
28/11/2026	Pitch e conclusão

7. Referências

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais. Brasília: Diário Oficial da União, 2018.

CAHN, Edgar S. No more throw-away people: the co-production imperative. 2. ed. Washington: Essential Books, 2004.

CAROLI, Paulo. Lean Inception: como alinhar pessoas e construir o produto certo. São Paulo: Caroli.org, 2018.

FLUTTER. Flutter documentation. Disponível em: https://docs.flutter.dev. Acesso em: 3 nov. 2026.

GOOGLE. Material Design 3: accessibility. Disponível em: https://m3.material.io/foundations/accessible-design. Acesso em: 3 nov. 2026.

INSTITUTO NACIONAL DE CONDOMÍNIOS E APOIO AOS CONDÔMINOS. Dados do setor condominial brasileiro. 2024.

JEST. Jest documentation: testing framework. Disponível em: https://jestjs.io/docs/getting-started. Acesso em: 3 nov. 2026.

NIELSEN, Jakob. 10 usability heuristics for user interface design. Nielsen Norman Group, 1994. Disponível em: https://www.nngroup.com/articles/ten-usability-heuristics. Acesso em: 3 nov. 2026.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS. Transformando nosso mundo: a Agenda 2030 para o Desenvolvimento Sustentável. Nova York: ONU, 2015.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation: transactions. Disponível em: https://www.postgresql.org/docs. Acesso em: 3 nov. 2026.

WORLD WIDE WEB CONSORTIUM. Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation, 2018. Disponível em: https://www.w3.org/TR/WCAG21. Acesso em: 3 nov. 2026.